

Query-Adaptive Subspace Projection for Inference-Time Jailbreak Defense

Jianyi Zhang*, Shizhao Liu, Zhen Li *Beijing Electronic Science and Technology Institute*
Beijing, Beijing, China
zjy@besti.edu.cn

Abstract

Safety-aligned large language models (LLMs) can still be coerced by jailbreak prompts into unsafe compliance. Existing defenses primarily operate at the input, output, or decoding level, while representation-level interventions often rely on a single fixed steering direction, making them brittle under prompt-dependent shifts in jailbreak behavior. We propose Query-Adaptive Subspace Projection (QSP), an inference-time jailbreak defense that represents safety-related and jailbreak-compliance signals as compact low-rank subspaces learned from contrastive calibration sets. QSP constructs safety and jailbreak subspace banks, selects an intervention layer, reduces concept interference through coupled subspace orthogonalization, and at inference time retrieves a query-adaptive Top- m basis according to subspace energy. It then applies dynamically scaled soft projection to suppress jailbreak-associated components without modifying model weights. Experiments on five open-source LLMs and eight prompt-only jailbreak families show that QSP provides effective detection and mitigation across diverse attack styles; on Mistral-7B, it reduces the average attack success rate from 0.76 without defense to 0.01, outperforming the strongest representation-level baseline. Our code is available at: <https://github.com/Carney-lsz/QSP/tree/main>.

1 Introduction

Large language models (LLMs) have revolutionized natural language processing, demonstrating remarkable capabilities in generation, reasoning, and interaction [7, 16, 29]. To mitigate risks of harmful outputs, modern LLMs are commonly equipped with safety alignment mechanisms, including constitutional AI [4] and reinforcement learning from human feedback (RLHF) [14, 30], that encourage refusals on unsafe requests. Despite these efforts, LLMs remain highly vulnerable to jailbreak attacks, where adversarial prompts bypass safety guardrails and elicit restricted content [26, 27, 46, 58]. Recent surveys further highlight the persistence and rapid

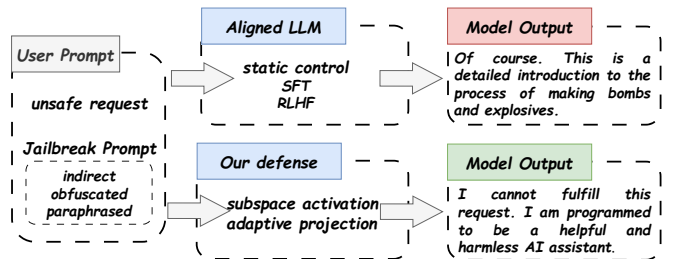


Figure 1: An illustration of the contrast between unsafe compliance caused by jailbreak prompts and safe refusal under our defense.

evolution of jailbreak strategies, including logits-based manipulation, optimization-driven suffixes, multi-turn social engineering, and multimodal variants [9, 11, 47, 53].

Unfortunately, existing defenses still fall short of robust and generalizable protection across attack families and prompt styles, as illustrated in Figure 1. On the one hand, input-level approaches such as perplexity filtering [20], prompt perturbation [8], or prompt smoothing [36] often rely on surface-form cues and become brittle under paraphrasing, obfuscation, and adaptive attacks, while also risking false positives on benign queries. Model-level hardening via fine-tuning [5, 6, 33] or unlearning [54] can improve robustness, yet frequently introduces utility degradation and substantial computational overhead. Hybrid strategies, including self-guard [42] and guardrail classifiers [13], show promise but still struggle with emerging multi-turn or heavily obfuscated jailbreaks [25, 45].

A promising direction is representation engineering [56], which interprets and manipulates hidden-state patterns to control model behavior. Prior studies suggest that many behavior-relevant factors exhibit low-dimensional and approximately linear structure in LLM representations [32]. Building on this line of work, JBSshield [52] identifies toxic and jailbreak concepts using mean-difference SVD vectors, enabling dual-activation detection and vector-steering mitigation. Subsequent works further investigate latent subspaces associated

with safe and jailbroken behaviors [12], and explore low-rank projection to stabilize aligned generation [41]. However, most prior methods still rely on single-vector interventions, fixed layer choices, and static intervention strength, which can be brittle across prompt lengths, attack families, and models. Moreover, two-stage designs that separately model safety and jailbreak signals may suffer from concept interference because the corresponding representation directions are not fully disentangled. This overlap couples their scores and can lead to unstable gating, resulting in either excessive mitigation on benign prompts or insufficient suppression under strong attacks.

In this paper, we revisit jailbreak behavior through a subspace-level lens and build an inference-time defense around three observations. First, safety-related semantics and jailbreak-induced compliance cues are more reliably characterized as low-dimensional subspaces rather than a single direction, and their overlap can cause cross-concept leakage if not explicitly handled. Second, successful jailbreaks consistently amplify a behavior-driving compliance factor that is not explained by safety semantics alone, and this factor often exhibits an early-token bias that steers the model toward compliance. Third, effective mitigation requires dynamic intervention, since a static always-on projection is prone to either damaging benign utility or failing under adaptive attacks.

Guided by these findings, we propose **QSP**, a unified inference-time jailbreak defense that combines subspace-based concept modeling with energy-guided dynamic projection control. Offline, our method constructs layer-wise contrastive difference matrices against benign references and applies unsupervised linear decomposition to obtain compact orthonormal concept subspaces. To reduce concept interference, our method introduces coupled subspace orthogonalization that removes safety-subspace components from the jailbreak subspace, yielding more stable signatures across prompt lengths, attack styles, and models. Online, our method detects jailbreaks using exponential moving average (EMA)-smoothed subspace energy signals and suppresses jailbreak-associated components via projection onto the complement subspace, while regulating projection strength through an energy-to-strength mapping so that intervention is activated and scaled only when warranted. We assume deployment-side access to hidden states, which is realistic for self-hosted models or integrated serving pipelines.

Across five target LLMs and eight jailbreak families, we achieve average detection accuracy exceeding 0.97 and reduce attack success rates (ASR) to under 3% while preserving benign utility. The main contributions of this work are as follows:

- We formulate jailbreak defense via orthonormal subspace analysis on calibrated contrast sets, and introduce coupled subspace orthogonalization to reduce concept interference and improve robustness across prompts, attacks, and models.
- We propose an energy-guided, dynamically regulated projection mechanism that suppresses jailbreak compliance cues while preserving benign utility by activating and scaling intervention based on smoothed subspace energies.
- We validate the full framework across five LLMs and eight jailbreak families, demonstrating strong detection performance and near-zero jailbreak success with negligible benign impact.

2 Related Work

2.1 Jailbreak Attacks and Conventional Defenses

Jailbreak attacks exploit misalignment in safety-tuned LLMs to elicit disallowed outputs. Early jailbreaks often rely on role-playing and instruction hijacking, and have evolved into optimization-based attacks such as GCG and AutoDAN that learn transferable adversarial suffixes across prompts and models [26, 58]. Recent variants further include multi-turn or conversational attacks that progressively shape the model’s internal state [45], as well as obfuscation and encoding-based strategies that bypass surface-form filters [24, 43]. Jailbreak techniques have also been extended to vision-language models, where typographic or compositional visual prompts can circumvent text-only safety alignment [17, 37].

Conventional defenses span prompt-stage filtering, input canonicalization, and generation-time safeguards. Input-stage defenses include perplexity-based screening [20], prompt perturbation [8], and randomized smoothing style strategies [36]. Other lines of work rely on prompt-based self-reminders, paraphrasing or retokenization, and output filtering, often aiming to reduce harmful completions without changing the base model. Training-based defenses such as refusal tuning and adversarial hardening can improve robustness, but they frequently introduce a safety utility tradeoff and add nontrivial training or deployment cost [5, 6, 33]. Self-defending frameworks that trigger model-internal checking show promising gains, yet remain challenged by sophisticated multi-turn or adaptive jailbreaks [25, 45].

2.2 Representation Engineering for LLM Control and Safety

Representation engineering controls model behavior by intervening on hidden states at inference time. A common approach is activation steering, which uses contrastive prompt pairs to derive a linear direction and then edits residual streams during generation [40, 56]. Subsequent work studies stronger and more robust interventions, including safety-oriented steering and hybrid schemes that combine steering with alignment objectives [31, 57]. These findings support

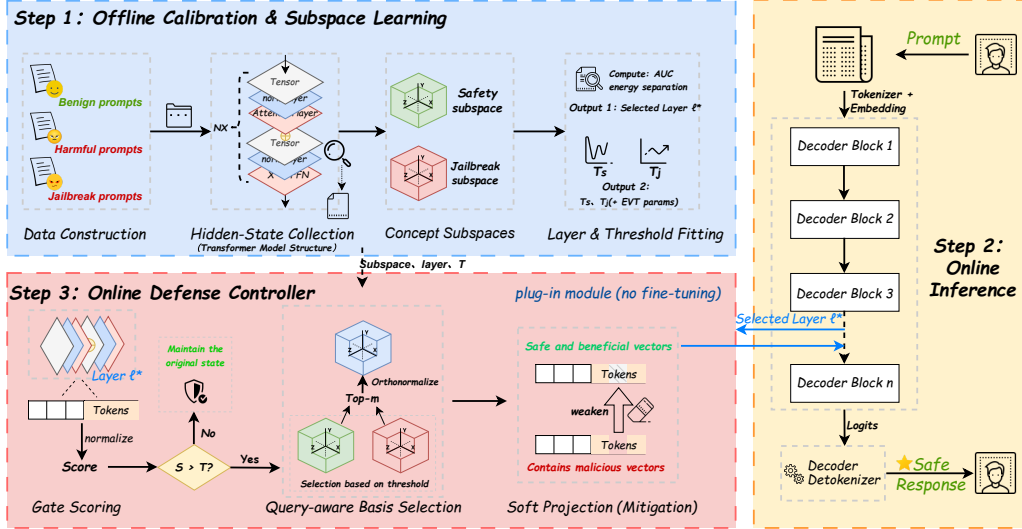


Figure 2: Overview of QSP. Step 1 learns safety and jailbreak subspace banks from calibration prompts, selects the intervention layer, and fits gating thresholds. Step 2 runs the input through the base LLM and extracts hidden states at the selected layer during decoding. Step 3 computes an energy-based gate and, when activated, retrieves query-relevant bases and applies soft projection to suppress jailbreak components before generation continues.

the broader view that behavior-relevant factors often admit low-dimensional structure in representation space.

For jailbreak-specific defense, JBSHield derives toxic and jailbreak concepts from contrastive hidden-state differences and applies dual-signal detection with vector arithmetic at selected layers [52]. Related analyses also show that refusal behavior can be strongly associated with low-dimensional directions, motivating representation-level edits for safety and robustness [3]. At the same time, many existing defenses still rely on a single editing direction, fixed layer selection, and static intervention strength. These design choices can be brittle under prompt-dependent concept shifts, and dual-stage designs may suffer from concept interference when safety and jailbreak representations overlap.

Projection-based concept removal has also been explored in broader concept erasure settings. However, systematic subspace-level projection that explicitly disentangles jailbreak compliance from safety semantics, and uses this disentanglement to stabilize detection and mitigation across diverse prompt styles, remains relatively limited.

Prior representation-level defenses suggest that jailbreak behavior leaves detectable traces in the hidden states of aligned LLMs. At the same time, their mixed robustness across prompt styles and attack families indicates that modeling such behavior with one or a few fixed steering directions may be fundamentally limited. This observation points to two broader insights. First, jailbreak-relevant behavior is better understood as a distributed semantic structure rather than an isolated direction, motivating a subspace-level view of representation-space intervention. Second, robust defense re-

quires disentangling jailbreak-induced compliance cues from overlapping safety-related semantics, since interference between these signals can blur decision boundaries and destabilize intervention. Guided by these insights, our method adopts subspace-level modeling and energy-guided adaptive control, yielding more stable activation signals, more precise intervention, and stronger generalization across diverse jailbreak forms while better preserving benign utility.

3 Threat Model

Setting. We study an aligned LLM deployed as a chat assistant. The defender seeks to prevent the model from producing policy-violating content while maintaining high utility on benign requests. Our defense runs *at inference time* with white-box access to hidden states in the serving pipeline, enabling detection and representation-space control.

Adversary. The adversary interacts with the system through the standard user interface and aims to induce unsafe compliance via *prompt-only* jailbreaks. The adversary can issue arbitrary inputs, including multi-turn conversations that gradually escalate intent, role-play instructions that attempt to override the system’s safety constraints, obfuscated prompts that conceal toxic semantics through encoding or language transformations, and transferable suffix-style prompts produced by automated or optimization-based procedures.

Success criterion. An attack succeeds if the model produces responses that materially advance the unsafe goal rather than refusing or safely redirecting. We report attack success using an automatic judge solely for evaluation.

Defender. The defender cannot modify the base model weights and instead relies on lightweight runtime mechanisms. Offline calibration over benign, harmful, and jailbreak prompts is allowed to derive concept vectors or low-rank subspaces and to set decision thresholds. At runtime, the defender monitors hidden representations and applies projection-based interventions during generation when malicious behavior is detected.

Out of scope. We exclude compromises that bypass the chat interface, including manipulation of deployment infrastructure, privileged leakage or modification of the system prompt, and model supply-chain attacks such as poisoned or backdoored checkpoints. We also exclude side-channel attacks and hardware-level attacks, and focus on plain text interactions.

4 Method

Overview. Our method is a plug-and-play, inference-time defense that detects and suppresses jailbreak-induced compliance by intervening on hidden states through *query-adaptive low-rank projection*, as illustrated in Figure 2. Specifically, the framework follows an offline-to-online pipeline: in Step 1, it constructs compact safety and jailbreak concept subspace banks from calibration prompts, selects the intervention layer, and calibrates the gating thresholds; in Step 2, the input is processed by the base LLM during decoding while hidden states at the selected layer are exposed for monitoring; in Step 3, a lightweight online controller scores the current representation and, when activated, performs gated query-adaptive basis selection and *soft projection* to suppress jailbreak-associated components before generation continues.

4.1 Notation and Problem Setup

Given an input prompt x , we denote the hidden state at layer ℓ as $H^{(\ell)}(x) \in \mathbb{R}^{S \times d}$, and the last-token hidden state as $h^{(\ell)}(x) = H^{(\ell)}(x)[S, :] \in \mathbb{R}^d$. We use $\mu^{(\ell)} \in \mathbb{R}^d$ as a benign reference mean at layer ℓ . We define the centered and normalized direction

$$\tilde{d}^{(\ell)}(x) = \text{norm}\left(h^{(\ell)}(x) - \mu^{(\ell)}\right), \quad \text{norm}(v) = \frac{v}{\|v\|_2 + \epsilon}. \quad (1)$$

We set ϵ as a small constant for numerical stability.

We consider two concept families: **Safety/toxicity** signals that correlate with harmful intent, and **Jailbreak/compliance** signals that correlate with bypassing refusal behavior. Instead

Algorithm 1 QSP Offline

Require: $\mathcal{D}_{\text{ben}}, \mathcal{D}_{\text{harm}}, \mathcal{D}_{\text{jb}}$; layers $1..L$, bank size k ; rank budget m

Ensure: $\{\mu^{(\ell)}\}; \{B_s^{(\ell)}, B_j^{(\ell)}\}$; selected layer ℓ^* ; thresholds (T_s, T_j)

```

1: for  $\ell = 1$  to  $L$  do
2:    $\mu^{(\ell)} \leftarrow \mathbb{E}_{x \in \mathcal{D}_{\text{ben}}} [h^{(\ell)}(x)]$ 
3:    $\mathcal{V}_s \leftarrow \{\text{norm}(h^{(\ell)}(x) - \mu^{(\ell)}) : x \in \mathcal{D}_{\text{harm}}\}$ 
4:    $\mathcal{V}_j \leftarrow \{\text{norm}(h^{(\ell)}(x) - \mu^{(\ell)}) : x \in \mathcal{D}_{\text{jb}}\}$ 
5:    $B_s^{(\ell)} \leftarrow \text{PCA}(\mathcal{V}_s, k); \quad B_j^{(\ell)} \leftarrow \text{PCA}(\mathcal{V}_j, k)$ 
6:    $Q_s^{(\ell)} \leftarrow \text{QR}(B_s^{(\ell)}[1:m]); \quad Q_j^{(\ell)} \leftarrow \text{QR}(B_j^{(\ell)}[1:m])$ 
7:    $Q_j^{(\ell)} \leftarrow \text{QR}((I - Q_s^{(\ell)} Q_s^{(\ell)\top}) Q_j^{(\ell)})$ 
8:   compute  $s_s^{(\ell)}, s_j^{(\ell)}$ 
9:    $\text{AUC}^{(\ell)} \leftarrow \text{AUC}(s_j^{(\ell)}; \mathcal{D}_{\text{jb}}, \mathcal{D}_{\text{ben}})$ 
10: end for
11:  $\ell^* \leftarrow \arg \max_{\ell} \text{AUC}^{(\ell)}$ 
12: return  $\{\mu^{(\ell)}\}, \{B_s^{(\ell)}, B_j^{(\ell)}\}, \ell^*, (T_s, T_j)$ 

```

of using a single steering vector, our method models each concept as a *low-dimensional subspace*.

Dimension and Rank Definitions. In our approach, we denote the dimension of the hidden states as d , which corresponds to the number of features in the model’s representation at each layer. The rank of the concept subspace is defined as r , which is typically much smaller than the full dimension, i.e., $r \ll d$.

Index Set Definition. We define I as the set of indices of the top- m basis vectors selected from the bank $B^{(\ell)}$ based on cosine similarity with the query vector q .

4.2 Offline Phase

Calibration sets. We assume access to a small calibration set containing harmless prompts $\mathcal{D}_{\text{ben}}$, harmful prompts $\mathcal{D}_{\text{harm}}$, and jailbreak prompts $\mathcal{D}_{\text{jb}}$. We only use these data *offline* to learn subspaces and thresholds; the base model weights remain frozen. We summarize the complete offline calibration procedure in Algorithm 1.

Subspace bank construction via PCA. For each layer ℓ , we collect normalized directions $\tilde{d}^{(\ell)}(x)$ from the corresponding positive set (e.g., $\mathcal{D}_{\text{harm}}$ for safety, or $\mathcal{D}_{\text{jb}}$ for jailbreak). We then apply principal component analysis (PCA) on these vectors to obtain a compact basis bank $B^{(\ell)} \in \mathbb{R}^{k \times d}$, where each row is a basis direction and $k \ll d$. We later convert a selected subset of rows into an orthonormal basis via QR.

Energy score. Given an orthonormal basis $Q^{(\ell)} \in \mathbb{R}^{d \times r}$ (columns orthonormal), we define an energy score:

$$s^{(\ell)}(x; Q) = \left\| \tilde{d}^{(\ell)}(x)^\top Q^{(\ell)} \right\|_2. \quad (2)$$

Intuitively, s measures how strongly the prompt activates the concept subspace.

Layer selection by energy separation. Concept activation can drift across models and prompts, so we select an intervention layer ℓ^* by maximizing separability. Concretely, for each concept we compute per-layer area under the curve (AUC) between positives and negatives using $s^{(\ell)}$, and choose the top layer(s) with highest energy separation. This yields the selected safety layer and jailbreak layer used by the runtime controller. In practice, we compute per-layer separability using the energy scores in Eq. (2) and select ℓ^* by maximizing AUC.

Threshold calibration with tail modeling. To obtain stable thresholds under distribution shift, we calibrate concept-specific gates from the benign score distribution, using extreme value theory (EVT) tail fitting when appropriate. We set (T_s, T_j) to meet a target false-positive rate on $\mathcal{D}_{\text{ben}}$, while enforcing high recall on the harmful/jailbreak calibration sets. The resulting (T_s, T_j) are recorded as offline outputs and reused by the online controller.

Coupled subspace orthogonalization. Two-stage defenses can suffer from concept interference when the safety subspace overlaps with the jailbreak subspace. To reduce leakage, we explicitly orthogonalize the jailbreak subspace against the safety subspace:

$$Q_j^{(\ell)} \leftarrow \text{QR} \left(\left(I - Q_s^{(\ell)} Q_s^{(\ell)\top} \right) Q_j^{(\ell)} \right), \quad (3)$$

where $\text{QR}(\cdot)$ denotes re-orthonormalization. This makes the jailbreak score and mitigation more specific, stabilizing gating and reducing benign utility loss. Appendix A.4 provides geometric diagnostics and interpretability case studies, including token-level concept traces and principal-angle analysis of coupled orthogonalization.

4.3 Online Phase

At runtime, we attach forward hooks at the selected layer and performs gate scoring, query-aware basis selection, and soft projection. We summarize the complete online control procedure in Algorithm 2.

4.3.1 Gate Scoring

Given the current hidden state $H^{(\ell^*)}(x)$, we compute an energy score s by Eq. (2). To reduce jitter across tokens and multi-

Algorithm 2 QSP Online

Require: layer ℓ^* (or window $\ell^* + \Omega$), bank $B^{(\ell)}$, mean $\mu^{(\ell)}$, gate (T, β) , rank m , strength $(\alpha_{\min}, \alpha_{\max}, \tau)$
Ensure: write-back mitigated hidden states $H^{(\ell)}$ when the gate triggers

- 1: **for** each decoding step t **do**
- 2: **for** each monitored layer $\ell \in \{\ell^* + \delta : \delta \in \Omega\}$ **do**
- 3: read hidden states $H^{(\ell)}$
- 4: $q \leftarrow \text{norm}(\text{LAST}(H^{(\ell)}) - \mu^{(\ell)})$
- 5: $s \leftarrow \|q^\top Q^{(\ell)}\|_2$
- 6: $\bar{s} \leftarrow \beta \bar{s} + (1 - \beta)s$
- 7: **if** $\bar{s} \leq T$ **then continue**
- 8: **end if**
- 9: $I \leftarrow \text{TopM}(\langle q, B^{(\ell)} \rangle, m)$
- 10: $Q \leftarrow \text{QR}(B^{(\ell)}[I])$
- 11: $\alpha \leftarrow \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot \sigma((\bar{s} - T)/\tau)$
- 12: $\Delta \leftarrow H^{(\ell)} - \mu^{(\ell)}$
- 13: $H^{(\ell)} \leftarrow \Delta - \alpha(\Delta Q Q^\top) + \mu^{(\ell)}$
- 14: **end for**
- 15: **end for**

turn context, we apply EMA smoothing:

$$\bar{s}_t = \beta \bar{s}_{t-1} + (1 - \beta)s_t. \quad (4)$$

The gate triggers if $\bar{s}_t > T$ (with separate thresholds for safety/jailbreak stages). This yields a low-overhead runtime detector that does not require additional models.

4.3.2 Query-aware Basis Selection

A fixed subspace can over-project benign directions or miss query-specific jailbreak variants. We therefore adapt the projection subspace per input by selecting a Top- m subset from the offline bank. Let $B^{(\ell^*)} = \{b_i\}_{i=1}^k$ be the bank (rows). We form a query direction from the last-token representation $h^{(\ell^*)} = \text{LAST}(H^{(\ell^*)})$:

$$q = \text{norm}(h^{(\ell^*)} - \mu^{(\ell^*)}), \quad (5)$$

and select the m basis vectors with largest cosine similarity $\langle q, b_i \rangle$. Then we orthonormalize the selected set via QR to obtain $Q \in \mathbb{R}^{d \times r}$. This makes the intervention *input-adaptive* while keeping the rank small ($r \leq m \ll d$).

4.3.3 Soft Projection and Write-back

We suppress jailbreak components by projecting the centered hidden states onto the orthogonal complement of Q :

$$\Delta = H^{(\ell^*)} - \mu^{(\ell^*)}, \quad \Delta' = \Delta - \alpha \cdot \text{Proj}_Q(\Delta), \quad (6)$$

where $\text{Proj}_Q(\Delta) = \Delta Q Q^\top$ and $\alpha \in [0, 1]$ controls strength. Finally, we write back $H^{(\ell^*)} \leftarrow \Delta' + \mu^{(\ell^*)}$. We use a *soft* projection (scaled by α) rather than hard removal to preserve

fluency and benign utility. We implement this intervention by directly writing back the modified hidden states (Algorithm 2). Appendix A.2 analyzes sensitivity to the projection strength α and the retrieved rank m .

Confidence-aware strength mapping. Instead of a static α , we map gate confidence to strength using a sigmoid:

$$\alpha = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot \sigma\left(\frac{\bar{s} - T}{\tau}\right), \quad (7)$$

where $\sigma(z) = \frac{1}{1 + \exp(-z)}$ is the logistic sigmoid, and τ is a temperature. This yields continuous control: near-threshold cases are minimally perturbed, while strong jailbreak activations are aggressively suppressed.

Multi-layer analysis window. To tolerate layer drift across prompts/models, we optionally apply the same projection logic to a small window $\{\ell^* + \delta : \delta \in \Omega\}$ (e.g., $\Omega = \{-1, 0, 1\}$), which empirically improves robustness with negligible overhead.

Early-token coverage scheduling. Optimization-driven attacks often steer behavior early and maintain it via long-range context. We therefore increase early-token coverage by projecting all tokens during an initial window of forward calls, and then periodically refreshing: this helps counter early steering while keeping runtime cost bounded.

5 Experiments

5.1 Datasets

We curate our evaluation corpus from three complementary input types: benign, harmful, and jailbreak prompts. For benign inputs, we sample 850 instruction-following queries from Alpaca-style[39] data, aiming to reflect everyday user requests across diverse scenarios. For harmful inputs, we combine 520 restricted-content prompts from AdvBench[58] with 330 prompts from HEx-PHI[34], yielding a unified pool of explicitly unsafe goals. To reduce bias from class imbalance, we downsample the benign pool to match the harmful pool size in all experiments.

For jailbreak inputs, we construct attack-conditioned prompts by running eight prompt-only jailbreak methods on five target LLMs under their default configurations. Concretely, we include in-the-wild jailbreak prompts released by prior work when available, and generate the remaining jailbreak instances by applying each attack to our harmful goals to produce jailbreak-style variants that attempt to elicit unsafe compliance. Overall, this procedure yields 34000 jailbreak prompts aggregated across models and attacks.

For each prompt pool, we randomly split the data into a calibration set and a test set. The calibration split is used

exclusively for offline design choices, including selecting intervention layers, estimating representation statistics, constructing low-rank safety and jailbreak subspaces, and tuning gating thresholds and projection hyperparameters. All reported results are computed on the held-out test split only, and we strictly prevent any test prompts from being used during calibration to avoid data leakage.

5.2 Experimental Setup

Target models. We benchmark five widely used LLMs spanning multiple model families: **Mistral-7B**[23], **Vicuna-7B**, **Llama-3-8B**[18], **Qwen2.5-7B**[35], and **Deepseek-7B**[15]. For reproducibility, we use the official HuggingFace checkpoints and each model’s default chat template; all methods share the same decoding setup and an identical system prompt.

Attack Methods. In our experimental setup, we evaluate a diverse set of representative prompt only jailbreak attacks to comprehensively stress test the proposed defense under different attack paradigms. Specifically, we include eight attacks and group them into five categories. The manually designed attack is IJP [38]. Optimization based attacks include SAA [2] and I-GCG [22]. Linguistics based attacks include DrAttack [25] and Puzzler [10]. Encoding based attacks include Base64 [43] and Zulu [51]. Obfuscation based attacks are represented by Flip [28].

Baselines. Detection. We benchmark against representative prompt stage detectors that identify jailbreak attempts before generation, covering multiple detection paradigms. These include an external toxicity classifier PAPI, a perplexity based detector PPL[1], a guard model classifier LlamaG[19], a self checking baseline Self Ex[42], a gradient or internal signal based method GradSafe[48], and JBShield[52], which detects jailbreak inputs via activation concepts. **Mitigation.** We further compare against representative generation time defenses that mitigate jailbreaks during decoding. We report No def as an unprotected reference. We include a reminder based method Self-Re[49], an input rewriting baseline PR[21], context based defenses such as ICD[44], a smoothing style defense SD[50], an optimization based baseline DRO[55], and JBShield[52], which performs representation level intervention by strengthening safety related concepts and suppressing jailbreak related concepts. We use the official implementations released by the authors and follow their recommended default hyperparameters for all baselines.

Metrics. Detection. We evaluate jailbreak input detection as a binary classification task that distinguishes jailbreak prompts from benign prompts. For each model and each attack family, we report **Accuracy** and **F1-score**, where F1 treats

Table 1: Performance of different jailbreak detection methods.

Model	Method	Accuracy↑ / F1-Score↑								Avg. Accuracy↑
		IJP	I-GCG	SAA	DrAttack	Puzzler	Zulu	Base64	Flip	
<i>Mistral-7B</i>	PAPI	0.04/0.08	0.04/0.08	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.01
	PPL	0.01/0.03	0.31/0.46	0.00/0.00	0.00/0.00	0.00/0.00	0.95/0.95	0.00/0.00	0.02/0.03	0.16
	LlamaG	0.68/0.81	0.75/0.84	0.83/0.90	0.84/0.91	0.77/0.87	0.50/0.67	0.58/0.73	0.72/0.84	0.71
	Self-Ex	0.42/0.59	0.49/0.65	0.40/0.57	0.51/0.67	0.44/0.62	0.32/0.49	0.37/0.54	0.45/0.62	0.43
	GradSafe	0.01/0.02	0.60/0.74	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.02/0.04	0.08
	JBShield	0.84/0.86	0.94/0.94	0.97/0.98	0.82/0.80	0.96/0.95	0.97/0.99	0.96/0.99	0.91/0.92	0.92
	Ours	0.88/0.89	0.96/0.95	0.96/0.95	0.99/0.99	1.00/1.00	0.97/0.97	0.96/0.96	1.00/1.00	0.96
<i>Vicuna-7B</i>	PAPI	0.04/0.08	0.12/0.23	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.02
	PPL	0.01/0.03	0.44/0.59	0.00/0.00	0.00/0.00	0.00/0.00	0.85/0.92	0.00/0.00	0.01/0.02	0.16
	LlamaG	0.65/0.79	0.72/0.83	0.85/0.91	0.84/0.91	0.75/0.86	0.49/0.65	0.55/0.71	0.70/0.83	0.69
	Self-Ex	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.01/0.02	0.01/0.03	0.00/0.00	0.00
	GradSafe	0.03/0.06	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.01/0.02	0.01
	JBShield	0.82/0.83	0.92/0.93	0.99/0.99	0.99/0.99	1.00/0.91	0.99/0.99	1.00/1.00	0.94/0.94	0.96
	Ours	0.96/0.96	0.98/0.97	0.99/0.99	0.97/0.96	1.00/1.00	0.93/0.92	1.00/1.00	0.99/0.99	0.98
<i>Llama3-8B</i>	PAPI	0.04/0.08	0.01/0.03	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.01
	PPL	0.01/0.03	0.82/0.87	0.00/0.00	0.00/0.00	0.00/0.00	0.95/0.95	0.00/0.00	0.01/0.02	0.22
	LlamaG	0.46/0.63	0.51/0.67	0.71/0.83	0.70/0.82	0.55/0.71	0.34/0.51	0.38/0.56	0.50/0.68	0.52
	Self-Ex	0.15/0.26	0.10/0.19	0.19/0.31	0.16/0.27	0.18/0.30	0.12/0.21	0.14/0.24	0.13/0.23	0.15
	GradSafe	0.41/0.58	0.18/0.32	0.00/0.00	0.00/0.00	0.92/0.96	0.00/0.00	0.00/0.00	0.02/0.05	0.19
	JBShield	0.91/0.92	0.95/0.96	0.98/0.97	0.97/0.96	0.99/0.99	0.96/0.94	0.97/0.97	0.91/0.94	0.96
	Ours	0.89/0.88	0.98/0.97	0.97/0.96	0.96/0.95	1.00/0.95	0.95/0.98	0.99/0.99	0.98/0.96	0.97
<i>Qwen2.5-7B</i>	PAPI	0.04/0.08	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.01
	PPL	0.01/0.03	0.79/0.86	0.00/0.00	0.00/0.00	0.00/0.00	0.10/0.18	0.00/0.00	0.00/0.00	0.11
	LlamaG	0.41/0.57	0.32/0.48	0.63/0.77	0.57/0.72	0.49/0.65	0.38/0.55	0.35/0.51	0.53/0.69	0.46
	Self-Ex	0.31/0.33	0.28/0.32	0.36/0.39	0.32/0.35	0.24/0.27	0.27/0.31	0.29/0.32	0.27/0.30	0.29
	GradSafe	0.39/0.56	0.97/0.98	0.00/0.00	0.00/0.00	0.18/0.31	0.96/0.98	0.00/0.00	0.62/0.77	0.39
	JBShield	0.82/0.83	0.89/0.82	0.90/0.92	0.96/0.96	0.92/0.82	0.95/0.95	0.92/0.90	0.85/0.88	0.90
	Ours	0.88/0.87	1.00/0.99	0.97/0.97	0.96/0.99	0.99/0.95	0.96/0.93	0.93/0.93	0.96/0.94	0.96
<i>Deepseek-7B</i>	PAPI	0.04/0.08	0.02/0.04	0.00/0.00	0.00/0.00	0.00/0.00	0.02/0.04	0.02/0.04	0.00/0.00	0.01
	PPL	0.01/0.03	0.85/0.90	0.00/0.00	0.00/0.00	0.00/0.00	0.23/0.36	0.00/0.00	0.00/0.00	0.14
	LlamaG	0.46/0.63	0.54/0.70	0.71/0.83	0.70/0.82	0.55/0.71	0.50/0.67	0.38/0.56	0.60/0.75	0.56
	Self-Ex	0.15/0.26	0.12/0.21	0.19/0.31	0.16/0.27	0.18/0.30	0.11/0.19	0.14/0.24	0.16/0.26	0.15
	GradSafe	0.41/0.58	0.21/0.35	0.00/0.00	0.00/0.00	0.92/0.96	0.97/0.98	0.00/0.00	0.37/0.54	0.36
	JBShield	0.88/0.90	0.95/0.89	0.98/0.97	0.95/0.95	0.97/0.93	0.96/0.97	0.90/0.92	0.91/0.93	0.95
	Ours	0.92/0.92	0.99/0.99	0.96/0.96	0.99/0.98	0.99/0.87	0.97/0.97	0.91/0.91	0.96/0.94	0.96

jailbreak as the positive class. We also report the **average accuracy** by taking the mean accuracy over the eight attacks for each model, as summarized in Table 1.

Mitigation. We measure defense effectiveness using **attack success rate**, defined as the fraction of jailbreak prompts for which the model response complies with the unsafe request. We use a fine tuned safety judge in the sorry bench style

that outputs 1 for unsafe compliance and 0 otherwise, and compute ASR as the mean judge score over the evaluation set. We report per attack ASR and the **average ASR** across the eight attacks for each model, as shown in Table 2.

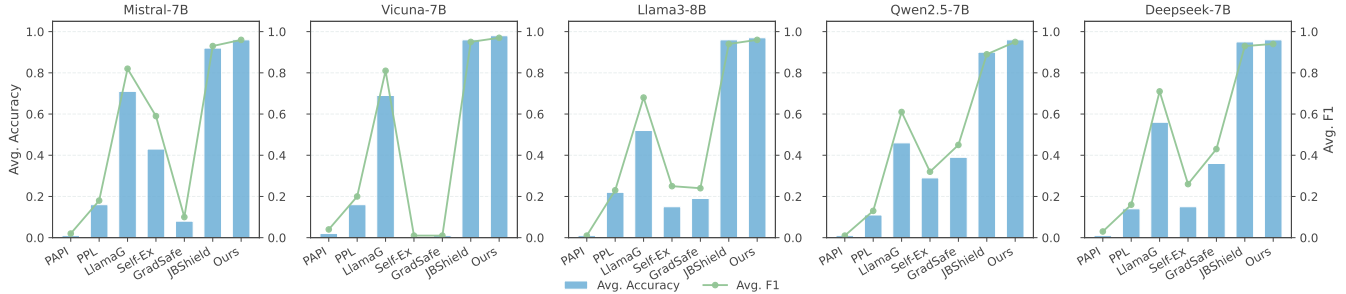


Figure 3: Performance of our method across multiple models.

Implementation details. Our experiments are based on open-source LLMs and implemented in PyTorch. We use forward hooks to access intermediate hidden states. Our method includes an offline calibration stage. It extracts layer-wise hidden states from calibration prompts, builds per-layer subspace banks for safety and jailbreak concepts, selects high-signal layers by energy-based AUC, and calibrates gating thresholds using EVT or percentile rules. When enabled, we further learn a lightweight 2D fusion gate via logistic regression over the safety score and the jailbreak score. At inference time, our method registers hooks on the selected layers, computes gate scores on-the-fly, retrieves a query-adaptive Top- m basis from the corresponding subspace bank, and applies soft projection to suppress unsafe directions in hidden states before generation continues. We enable multi-layer window projection, coupled orthogonalization between the safety and jailbreak subspaces, and token-coverage schedules for stronger attacks. Additional implementation details are provided in Appendix A.1.

5.3 Detection Results

Table 1 reports per-attack Accuracy and F1 for jailbreak-input detection across five LLM backbones and eight prompt-only attack families. Overall, our method achieves strong and stable average accuracy, consistently outperforming prompt-stage baselines that mainly rely on surface-form cues. Toxicity- and perplexity-based detectors such as *PAPI* and *PPL* exhibit high variance across attack families. *PAPI* often yields low F1 on many attacks and can approach a random-guess or majority baseline when class balance is skewed. *PPL* performs well primarily on highly non-natural prompts, including optimization-style adversarial suffixes such as *I-GCG* and heavily perturbed or encoded inputs such as *Zulu*, but its performance drops substantially on other families, indicating limited cross-attack generalization. In contrast, our method maintains robust performance on optimization-style suffix attacks such as *I-GCG*, structured or indirect attacks such as *Puzzler* and *DrAttack*, and obfuscation or encoding attacks including *Flip*, *Zulu*, and *Base64*, where lexical heuristics and fluency signals are intrinsically unreliable. Figure 3 further

summarizes methods average Accuracy and F1 across attacks and highlights a clear stability gap between prompt-stage heuristics and representation-level detectors.

Compared to the strongest representation-level baseline *JBSHield-D*, our method remains competitive overall. On *Mistral-7B* and *Vicuna-7B*, the average metrics are typically within a small margin, and on *Llama3* and *Qwen* we match or improve upon its performance. Beyond the averages, we observe clearer gains on hard cases where benign-like phrasing or indirect cues weaken single-vector concept gating. For instance, our method shows marked improvements on *DrAttack* and *Flip* under *Mistral-7B* and achieves stronger F1 on difficult structured prompts such as *Puzzler* on *Vicuna-7B*. These improvements are driven by higher recall on borderline jailbreak inputs rather than superficial gains from trivial majority-class prediction.

These trends align with our design choices. Energy-based subspace scoring aggregates multiple correlated jailbreak directions, which makes the detector less sensitive to paraphrasing, obfuscation, and token-level perturbations than cosine gating to a single direction. We further model two complementary energy signals, one computed in a safety subspace and the other computed in a jailbreak subspace, and fuse them to separate harmful intent from jailbreak-style steering. This separation improves robustness when attacks reduce explicit toxicity while still inducing latent compliance cues.

5.4 Mitigation Results

Table 2 reports ASR under eight prompt-only jailbreak families across five open-source backbones. Overall, our method consistently suppresses jailbreak success to a near-zero level and establishes the strongest robustness frontier among all evaluated defenses.

Without defense, all backbones remain highly vulnerable, confirming that safety alignment alone is insufficient against jailbreaks. After enabling our defense, jailbreak success is broadly suppressed across models, with several backbones exhibiting almost complete elimination of successful attacks. Compared to the strongest representation-level baseline *JBSHield-M*, our method further reduces the residual jailbreak

Table 2: Performance of different jailbreak mitigation methods. No-Def means no defense is deployed.

Models	Methods	Attack Success Rate↓								Average ASR↓
		IJP	I-GCG	SAA	DrAttack	Puzzler	Zulu	Base64	Flip	
<i>Mistral-7B</i>	No-def	0.56	0.95	0.98	0.74	1.00	0.48	0.40	0.94	0.76
	Self-Re	0.46	0.83	0.86	0.40	1.00	0.40	0.18	0.82	0.62
	PR	0.40	1.00	0.80	0.08	0.90	0.48	0.20	0.98	0.61
	ICD	0.52	0.48	0.58	0.68	1.00	0.06	0.08	0.46	0.48
	SD	0.52	0.73	0.96	0.86	1.00	0.32	0.40	0.75	0.69
	DRO	0.50	0.90	0.96	0.46	1.00	0.48	0.42	0.89	0.70
	JBShield	0.24	0.39	0.12	0.04	0.00	0.02	0.00	0.04	0.11
	Ours	0.08	0.00	0.00	0.00	0.00	0.02	0.00	0.00	0.01
<i>Vicuna-7B</i>	No-def	0.38	0.89	0.96	0.94	0.95	0.12	0.18	0.87	0.66
	Self-Re	0.34	1.00	0.88	0.62	0.95	0.18	0.00	0.98	0.62
	PR	0.22	1.00	0.82	0.34	0.80	0.40	0.22	0.98	0.60
	ICD	0.26	0.83	0.68	0.70	0.85	0.00	0.02	0.82	0.52
	SD	0.08	0.02	0.04	0.12	0.35	0.00	0.00	0.05	0.08
	DRO	0.36	1.00	0.64	0.52	0.95	0.54	0.06	0.99	0.63
	JBShield	0.04	0.21	0.00	0.00	0.00	0.00	0.00	0.02	0.05
	Ours	0.00	0.00	0.00	0.22	0.00	0.00	0.00	0.00	0.03
<i>Llama3-8B</i>	No-def	0.24	0.67	0.74	0.38	0.45	0.52	0.48	0.70	0.52
	Self-Re	0.02	0.18	0.44	0.36	0.00	0.02	0.00	0.20	0.15
	PR	0.26	0.13	0.14	0.04	0.05	0.46	0.06	0.15	0.16
	ICD	0.00	0.13	0.18	0.00	0.00	0.00	0.00	0.12	0.05
	SD	0.42	0.37	0.28	0.40	0.95	0.50	0.50	0.42	0.48
	DRO	0.24	0.23	0.42	0.12	0.00	0.60	0.14	0.25	0.25
	JBShield	0.00	0.02	0.00	0.00	0.00	0.02	0.00	0.01	0.01
	Ours	0.08	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
<i>Qwen2.5-7B</i>	No-def	0.38	0.99	0.96	0.94	0.95	0.12	0.18	0.28	0.60
	Self-Re	0.34	1.00	0.88	0.62	0.95	0.18	0.00	0.96	0.62
	PR	0.22	1.00	0.82	0.34	0.80	0.40	0.22	0.96	0.60
	ICD	0.26	0.83	0.68	0.70	0.85	0.00	0.02	0.82	0.52
	SD	0.08	0.02	0.04	0.12	0.35	0.00	0.00	0.05	0.08
	DRO	0.36	1.00	0.64	0.52	0.95	0.54	0.06	0.99	0.63
	JBShield	0.18	0.30	0.08	0.06	0.10	0.05	0.06	0.20	0.13
	Ours	0.08	0.00	0.00	0.00	0.00	0.04	0.00	0.00	0.02
<i>Deepseek-7B</i>	No-def	0.24	0.86	0.74	0.38	0.45	0.52	0.48	0.54	0.53
	Self-Re	0.02	0.18	0.44	0.36	0.00	0.02	0.00	0.20	0.15
	PR	0.26	0.13	0.14	0.04	0.05	0.46	0.06	0.15	0.16
	ICD	0.00	0.13	0.18	0.00	0.00	0.00	0.00	0.12	0.05
	SD	0.42	0.37	0.28	0.40	0.95	0.50	0.50	0.42	0.48
	DRO	0.24	0.23	0.42	0.12	0.00	0.60	0.14	0.25	0.25
	JBShield	0.05	0.08	0.02	0.02	0.03	0.02	0.03	0.06	0.04
	Ours	0.02	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

success on multiple backbones, while staying highly competitive on the remaining ones, indicating strong cross-model robustness rather than backbone-specific tuning.

Across attacks, a consistent pattern in Table 2 is that many input-side transformations or decoding-time heuristics may reduce some families, yet still fail on stronger optimization- or steering-driven suffix jailbreaks and structured-indirect

prompts. Defenses such as *Self-Re* and *PR* can leave substantial residual success on *I-GCG* and *Puzzler* across backbones, suggesting that shallow prompt manipulations do not reliably remove jailbreak-induced control signals. Context and smoothing-style defenses, such as *ICD* and *SD*, can help on certain obfuscation or encoding cases, but still struggle against strong steering attacks. In contrast, our method

Table 3: Ablation study of our method on two backbones. We remove three key online components and report per-attack ASR↓ and Average ASR↓.

Models	Variants	Attack Success Rate↓								Average ASR↓
		IJP	I-GCG	SAA	DrAttack	Puzzler	Zulu	Base64	Flip	
<i>Mistral-7B</i>	Full	0.08	0.00	0.00	0.00	0.00	0.02	0.00	0.00	0.01
	no-learned	0.25	0.30	0.18	0.12	0.28	0.10	0.08	0.13	0.18
	cosine	0.28	0.35	0.22	0.14	0.33	0.12	0.09	0.15	0.21
	no-coupling	0.28	0.25	0.28	0.31	0.30	0.28	0.26	0.32	0.29
<i>Llama3-8B</i>	Full	0.08	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01
	no-learned	0.21	0.29	0.18	0.12	0.31	0.17	0.07	0.11	0.17
	cosine	0.27	0.36	0.20	0.15	0.32	0.12	0.16	0.12	0.21
	no-coupling	0.34	0.28	0.28	0.32	0.26	0.36	0.28	0.37	0.31

consistently drives these hard families to near-zero success across all backbones, highlighting the necessity of suppressing representation-space compliance directions for robust mitigation.

These results are consistent with our design choices. Query-adaptive Top- m basis selection allows the intervention subspace to track prompt-dependent jailbreak variants, avoiding the brittleness of fixed-direction editing when control signals shift across suffix or template instantiations. Energy-guided gating operates on low-rank subspace activation rather than surface-form cues, which explains the robust behavior on obfuscation or encoding attacks where toxicity and perplexity signals are unreliable. Coupled orthogonalization reduces interference between safety-related and jailbreak-related components, enabling stronger projection with less collateral damage. Finally, token-coverage scheduling targets early-token steering by applying projection earlier and over a broader token scope, preventing the model from entering and sustaining a compliance trajectory. This mechanism aligns with the strong suppression observed on optimization-driven and structured jailbreaks.

5.5 Ablations

Table 3 isolates the key *online* decisions in our method, aiming to verify that the strong mitigation performance is not an artifact of calibration choices or prompt selection. We keep all calibration artifacts fixed, including the selected layers, base means, and subspace bases, and only remove one online controller component at a time. We report per-attack ASR and the average ASR across eight jailbreak families on two backbones, *Mistral-7B* and *Llama3-8B*.

What is ablated. **Learned fusion** removes the learned two-signal controller and falls back to a simple hand-crafted rule. **Cosine gate** replaces the subspace-energy score with a single-direction cosine score. **No coupling** disables the coupled orthogonalization between safety and jailbreak subspaces.

Analysis. The full system stays near-zero ASR on both backbones, indicating that the pipeline can consistently suppress a diverse set of prompt-only attacks rather than overfitting to a specific model or attack style. Once learned fusion is removed, the average ASR increases noticeably across most attacks on both backbones. This supports the design choice that the controller should jointly interpret the safety and jailbreak signals, instead of relying on a coarse trigger that treats them independently. In practice, many jailbreaks do not present as a single, clean signature; rather, they combine goal-shifting with subtle steering, and a learned boundary produces earlier and more stable intervention.

Replacing the energy gate with a cosine gate will lead to a decrease in performance. This indicates that cosine gating depends on alignment to a single direction and becomes brittle under paraphrasing, obfuscation, and optimization-driven suffix variations. By contrast, energy aggregation over a concept *subspace* provides a smoother score that is less sensitive to prompt surface form, which improves both detection stability and downstream projection quality. The effect is visible on attacks with heavy rewriting or encoding patterns, where cosine scores can fluctuate more.

Disabling **subspace coupling** also harms robustness, with failures concentrating on attacks where safety and jailbreak features are more entangled. Coupled orthogonalization matters because the defense is inherently compositional: a safety stage and a jailbreak stage are applied online, and their subspaces can partially overlap. Without coupling, the two stages may interfere, reducing the effective removal of jailbreak features and leaving a larger residual attack surface. The same trend holds on both *Mistral-7B* and *Llama3-8B*, suggesting the benefit is not model-specific but comes from making the two concept spaces better separated.

Overall, this ablation highlights that our method’s robustness primarily comes from *online control*, not from adding more offline artifacts. The learned fusion improves decision quality, subspace energy scoring improves stability under prompt variation, and coupling reduces interference between

the two stages. Together they explain why the full pipeline achieves consistently low ASR across backbones and across heterogeneous jailbreak families.

6 Discussion

Prior jailbreak defenses primarily rely on training based hardening, input canonicalization and filtering, or output moderation and self checking at generation time. While these approaches can reduce unsafe compliance, they often introduce a safety utility tradeoff, add nontrivial training or serving cost, and remain vulnerable to adaptive prompt only attacks that exploit paraphrasing, obfuscation, multi turn context, or transferable suffix patterns. Representation level methods offer an appealing inference time alternative, but many rely on a single steering direction and brittle gating rules, and can conflate safety related features with jailbreak induced compliance signals, leading to either insufficient suppression or excessive collateral damage on benign requests. Our method follows a different design point by modeling both safety and jailbreak compliance as low rank concept subspaces learned from calibrated contrast sets, selecting intervention layers via energy separation, and triggering a lightweight energy based controller with temporal smoothing. When activated, it performs query adaptive basis selection and applies confidence aware soft projection with multi layer windows and early token coverage scheduling, while explicitly orthogonalizing the jailbreak subspace against the safety subspace to reduce concept interference. This yields a plug in defense that does not modify base model weights and can be integrated into serving stacks with access to hidden states.

7 Limitations and Future Work

Our threat model assumes the defender can access hidden states in the serving pipeline and attach lightweight hooks for scoring and intervention. This is realistic for self-hosted or integrated deployments, but it may not apply to strictly black-box API settings. An important future direction is to explore whether similar control signals can be approximated through lightweight surrogates or distilled monitors when direct activations are unavailable.

8 Conclusion

We presented QSP, an inference time jailbreak defense that formulates concept manipulation as query adaptive low rank subspace projection in representation space. The approach separates offline calibration, where we learn compact concept subspace banks and select thresholds, from online control, where we perform energy based gating, query aware basis retrieval, and confidence aware soft projection. When stronger control is needed, the online mechanism also supports multi

layer intervention and token coverage schedules. This projection centric view expands the design space beyond fixed vector editing while remaining lightweight and deployable without modifying model weights. Our results across multiple open source LLM backbones and diverse prompt only jailbreak families suggest that subspace projection provides a practical and interpretable path toward controllable representation level jailbreak mitigation.

References

- [1] ALON, G., AND KAMFONAS, M. Detecting language model attacks with perplexity, 2023.
- [2] ANDRIUSHCHENKO, M., CROCE, F., AND FLAMMARION, N. Jailbreaking leading safety-aligned llms with simple adaptive attacks. *arXiv preprint arXiv:2404.02151* (2024).
- [3] ARDITI, A., OBESO, O., SYED, A., PALEKA, D., PANICKSSERY, N., GURNEE, W., AND NANDA, N. Refusal in language models is mediated by a single direction, 2024.
- [4] BAI, Y., KADAVATH, S., KUNDU, S., ASKELL, A., KERNION, J., JONES, A., CHEN, A., GOLDIE, A., MIRHOSEINI, A., MCKINNON, C., ET AL. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073* (2022).
- [5] BHARDWAJ, R., AND PORIA, S. Red-teaming large language models using chain of utterances for safety-alignment, 2023.
- [6] BIANCHI, F., SUZGUN, M., ATTANASIO, G., RÖTTGER, P., JURAFSKY, D., HASHIMOTO, T., AND ZOU, J. Safety-tuned llamas: Lessons from improving the safety of large language models that follow instructions, 2024.
- [7] BROWN, T., AND MANN, E. A. Language models are few-shot learners. In *Advances in Neural Information Processing Systems* (2020), H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, Curran Associates, Inc., pp. 1877–1901.
- [8] CAO, B., CAO, Y., LIN, L., AND CHEN, J. Defending against alignment-breaking attacks via robustly aligned llm, 2024.
- [9] CARLINI, N., NASR, M., CHOQUETTE-CHOO, C. A., JAGIELSKI, M., GAO, I., AWADALLA, A., KOH, P. W., IPPOLITO, D., LEE, K., TRAMER, F., AND SCHMIDT, L. Are aligned neural networks adversarially aligned?, 2024.
- [10] CHANG, Z., LI, M., LIU, Y., WANG, J., WANG, Q., AND LIU, Y. Play guessing game with llm: Indirect jailbreak attack with implicit clues. *arXiv preprint arXiv:2402.09091* (2024).
- [11] CHAO, P., ROBAY, A., DOBRIBAN, E., HASSANI, H., PAPPAS, G. J., AND WONG, E. Jailbreaking black box large language models in twenty queries, 2024.
- [12] CHIA, X. W., WONG, S. L., AND PAN, J. Probing latent subspaces in llm for ai security: Identifying and manipulating adversarial states. *arXiv preprint arXiv:2503.09066* (2025).
- [13] CHUA, G., CHAN, S. Y., AND KHOO, S. A flexible large language models guardrail development methodology applied to off-topic prompt detection. *arXiv preprint arXiv:2411.12946* (2024).
- [14] DAI, J., PAN, X., SUN, R., JI, J., XU, X., LIU, M., WANG, Y., AND YANG, Y. Safe rlhf: Safe reinforcement learning from human feedback, 2023.

- [15] DEEPSEEK-AI, :, BI, X., CHEN, D., CHEN, G., CHEN, S., DAI, D., DENG, C., DING, H., DONG, K., DU, Q., FU, Z., GAO, H., GAO, K., GAO, W., GE, R., GUAN, K., GUO, D., AND GUO., J. Deepseek llm: Scaling open-source language models with longtermism, 2024.
- [16] DEEPSEEK-AI. Deepseek-v3 technical report, 2025.
- [17] GONG, Y., RAN, D., LIU, J., WANG, C., CONG, T., WANG, A., DUAN, S., AND WANG, X. Figstep: Jailbreaking large vision-language models via typographic visual prompts, 2025.
- [18] GRATTAFIORI, A., DUBEY, A., JAHHRI, A., PANDEY, A., KADIAN, A., AND AHMAD AL-DAHLE, E. A. The llama 3 herd of models, 2024.
- [19] INAN, H., UPASANI, K., CHI, J., RUNGTA, R., IYER, K., MAO, Y., TONTCHEV, M., HU, Q., FULLER, B., TESTUGGINE, D., AND KHABSA, M. Llama guard: Llm-based input-output safeguard for human-ai conversations, 2023.
- [20] JAIN, N., SCHWARZSCHILD, A., WEN, Y., SOMEPALLI, G., KIRCHENBAUER, J., CHIANG, P.-Y., GOLDBLUM, M., SAHA, A., GEIPING, J., AND GOLDSTEIN, T. Baseline defenses for adversarial attacks against aligned language models. *arXiv preprint arXiv:2309.00614* (2023).
- [21] JAIN, N., SCHWARZSCHILD, A., WEN, Y., SOMEPALLI, G., KIRCHENBAUER, J., YEH CHIANG, P., GOLDBLUM, M., SAHA, A., GEIPING, J., AND GOLDSTEIN, T. Baseline defenses for adversarial attacks against aligned language models, 2023.
- [22] JIA, X., PANG, T., DU, C., HUANG, Y., GU, J., LIU, Y., CAO, X., AND LIN, M. Improved techniques for optimization-based jailbreaking on large language models, 2024.
- [23] JIANG, A. Q., SABLAYROLLES, A., MENSCH, A., BAMFORD, C., CHAPLOT, D. S., DE LAS CASAS, D., BRESSAND, F., LENGUEL, G., LAMPLE, G., SAULNIER, L., LAUDAUD, L. R., LACHAUX, M.-A., STOCK, P., SCAO, T. L., LAVRIL, T., WANG, T., LACROIX, T., AND SAYED, W. E. Mistral 7b, 2023.
- [24] KANG, D., LI, X., STOICA, I., GUESTRIN, C., ZAHARIA, M. A., AND HASHIMOTO, T. Exploiting programmatic behavior of llms: Dual-use through standard security attacks. *2024 IEEE Security and Privacy Workshops (SPW)* (2023), 132–143.
- [25] LI, X., WANG, R., CHENG, M., ZHOU, T., AND HSIEH, C.-J. Drat-tack: Prompt decomposition and reconstruction makes powerful llm jailbreakers. *arXiv preprint arXiv:2402.16914* (2024).
- [26] LIU, X., XU, N., CHEN, M., AND XIAO, C. Autodan: Generating stealthy jailbreak prompts on aligned large language models, 2024.
- [27] LIU, Y., DENG, G., XU, Z., LI, Y., ZHENG, Y., ZHANG, Y., ZHAO, L., ZHANG, T., WANG, K., AND LIU, Y. Jailbreaking chatgpt via prompt engineering: An empirical study, 2024.
- [28] LIU, Y., HE, X., XIONG, M., FU, J., DENG, S., AND HOOI, B. Flip-attack: Jailbreak llms via flipping, 2024.
- [29] OPENAI. Gpt-4 technical report, 2024.
- [30] OUYANG, L., WU, J., JIANG, X., ALMEIDA, D., WAINWRIGHT, C., MISHKIN, P., ZHANG, C., AGARWAL, S., SLAMA, K., RAY, A., ET AL. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [31] PANICKSSERY, N., GABRIELI, N., SCHULZ, J., TONG, M., HUBINGER, E., AND TURNER, A. M. Steering llama 2 via contrastive activation addition, 2024.
- [32] PARK, K., CHOE, Y. J., AND VEITCH, V. The linear representation hypothesis and the geometry of large language models. *arXiv preprint arXiv:2311.03658* (2023).
- [33] QI, X., ZENG, Y., XIE, T., CHEN, P.-Y., JIA, R., MITTAL, P., AND HENDERSON, P. Fine-tuning aligned language models compromises safety, even when users do not intend to! *arXiv preprint arXiv:2310.03693* (2023).
- [34] QI, X., ZENG, Y., XIE, T., CHEN, P.-Y., JIA, R., MITTAL, P., AND HENDERSON, P. Fine-tuning aligned language models compromises safety, even when users do not intend to!, 2023.
- [35] QWEN, :, YANG, A., YANG, B., ZHANG, B., HUI, B., ZHENG, B., YU, B., LI, C., AND DAYIHENG LIU, E. T. Qwen2.5 technical report, 2025.
- [36] ROBEY, A., WONG, E., HASSANI, H., AND PAPPAS, G. J. Smoothllm: Defending large language models against jailbreaking attacks. *arXiv preprint arXiv:2310.03684* (2023).
- [37] SHAYEGANI, E., DONG, Y., AND ABU-GHAZALEH, N. Jailbreak in pieces: Compositional adversarial attacks on multi-modal language models, 2023.
- [38] SHEN, X., CHEN, Z., BACKES, M., SHEN, Y., AND ZHANG, Y. "do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security* (2024), pp. 1671–1685.
- [39] TAORI, R., GULRAJANI, I., ZHANG, T., DUBOIS, Y., LI, X., GUESTRIN, C., LIANG, P., AND HASHIMOTO, T. B. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [40] TURNER, A. M., THIERGART, L., LEECH, G., UDELL, D., VAZQUEZ, J. J., MINI, U., AND MACDIARMID, M. Steering language models with activation engineering, 2024.
- [41] WANG, C., LYU, Y., SUN, Z., AND JING, L. Continual gradient low-rank projection fine-tuning for llms. *arXiv preprint arXiv:2507.02503* (2025).
- [42] WANG, Z., YANG, F., WANG, L., ZHAO, P., WANG, H., CHEN, L., LIN, Q., AND WONG, K.-F. SELF-GUARD: Empower the LLM to safeguard itself. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (Mexico City, Mexico, June 2024), K. Duh, H. Gomez, and S. Bethard, Eds., Association for Computational Linguistics, pp. 1648–1668.
- [43] WEI, A., HAGHTALAB, N., AND STEINHARDT, J. Jailbroken: How does llm safety training fail? *Advances in Neural Information Processing Systems* 36 (2023), 80079–80110.
- [44] WEI, Z., WANG, Y., LI, A., MO, Y., AND WANG, Y. Jailbreak and guard aligned language models with only few in-context demonstrations, 2024.
- [45] WENG, Z., JIN, X., JIA, J., AND ZHANG, X. Foot-in-the-door: A multi-turn jailbreak for llms. *arXiv preprint arXiv:2502.19820* (2025).
- [46] WHITE, J., FU, Q., HAYS, S., SANDBORN, M., OLEA, C., GILBERT, H., ELNASHAR, A., SPENCER-SMITH, J., AND SCHMIDT, D. C. A prompt pattern catalog to enhance prompt engineering with chatgpt, 2023.
- [47] XIE, P., BIE, Y., MAO, J., SONG, Y., WANG, Y., CHEN, H., AND CHEN, K. Chain of attack: On the robustness of vision-language models against transfer-based adversarial attacks, 2024.

- [48] XIE, Y., FANG, M., PI, R., AND GONG, N. Gradsafe: Detecting jailbreak prompts for llms via safety-critical gradient analysis, 2024.
- [49] XIE, Y., YI, J., SHAO, J., CURL, J., LYU, L., CHEN, Q., XIE, X., AND WU, F. Defending chatgpt against jailbreak attack via self-reminders. *Nature Machine Intelligence* 5 (2023), 1486–1496.
- [50] XU, Z., JIANG, F., NIU, L., JIA, J., LIN, B. Y., AND POOVENDRAN, R. Safedecoding: Defending against jailbreak attacks via safety-aware decoding, 2024.
- [51] YONG, Z.-X., MENGHINI, C., AND BACH, S. H. Low-resource languages jailbreak gpt-4. *arXiv preprint arXiv:2310.02446* (2023).
- [52] ZHANG, S., ZHAI, Y., GUO, K., HU, H., GUO, S., FANG, Z., ZHAO, L., SHEN, C., WANG, C., AND WANG, Q. Jbshield: Defending large language models from jailbreak attacks through activated concept analysis and manipulation. *arXiv preprint arXiv:2502.07557* (2025).
- [53] ZHANG, Z., SHEN, G., TAO, G., CHENG, S., AND ZHANG, X. Make them spill the beans! coercive knowledge extraction from (production) llms, 2023.
- [54] ZHANG, Z., YANG, J., KE, P., CUI, S., ZHENG, C., WANG, H., AND HUANG, M. Safe unlearning: A surprisingly effective and generalizable solution to defend against jailbreak attacks. *arXiv preprint arXiv:2407.02855* 1, 2 (2024), 3.
- [55] ZHENG, C., YIN, F., ZHOU, H., MENG, F., ZHOU, J., CHANG, K.-W., HUANG, M., AND PENG, N. On prompt-driven safeguarding for large language models, 2024.
- [56] ZOU, A., PHAN, L., CHEN, S., CAMPBELL, J., GUO, P., REN, R., PAN, A., YIN, X., MAZEIKA, M., DOMBROWSKI, A.-K., ET AL. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405* (2023).
- [57] ZOU, A., PHAN, L., WANG, J., DUENAS, D., LIN, M., ANDRIUSHCHENKO, M., WANG, R., KOLTER, Z., FREDRIKSON, M., AND HENDRYCKS, D. Improving alignment and robustness with circuit breakers, 2024.
- [58] ZOU, A., WANG, Z., CARLINI, N., NASR, M., KOLTER, J. Z., AND FREDRIKSON, M. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).

A Additional Details and Experiments

All experiments that require LLM inference are run on a single workstation equipped with an NVIDIA GeForce RTX 4090 GPU (24GB VRAM). Due to resource constraints, we report results from a single end-to-end run; to ensure reproducibility, we fix random seeds and use stable generation settings so the outcomes are consistent across reruns.

A.1 Implementation Details

Models and decoding. We evaluate multiple open source instruction tuned backbones. All generations use a fixed decoding configuration across defenses and baselines, including maximum new tokens, temperature, and top p. We use the same decoding for defended and undefended runs to isolate the effect of inference time interventions.

ASR judging protocol. We measure jailbreak robustness using attack success rate, where an attempt is successful if the model output is judged as non refusal and harmful. We adopt an automatic judge model finetuned for refusal and safety classification, following the standard practice in recent jailbreak benchmarks. We use a fixed judge prompt and thresholding rule across all methods to ensure comparability.

Offline calibration. Our offline stage learns compact concept subspace banks and gating thresholds. For each candidate layer, we extract hidden representations under calibration prompts and compute class conditional statistics. We select intervention layers by maximizing separability between target classes using a held out calibration fold. For each selected layer, we compute a low rank basis using PCA on centered representations, and retain the top- k components as the concept subspace bank. We calibrate gating thresholds on the calibration split to meet a target false positive rate or a target recall floor, depending on the gate type. For the two stage variant, we further orthogonalize the jailbreak subspace against the safety subspace to reduce interference when both stages trigger.

Online control and projection. At inference time, we extract hidden states from the selected layers. For each token, we compute an energy-based gating score, smooth it with an EMA, and compare it against calibrated thresholds. Once the gate is activated, we retrieve the top- m bases from the concept bank via query-adaptive similarity, and apply a confidence-aware soft projection by subtracting the component along the retrieved bases. The projection strength is set by a smooth mapping from the gate score to α , and can be further scheduled with a token-coverage policy that emphasizes early tokens. For multi-layer projection, we apply the same online procedure within a small window around each selected layer, reusing the calibrated bank.

A.2 Hyperparameter Sensitivity: Top- m and Projection Strength

What needs to be validated. Our method has two inference-time knobs that directly control the robustness–utility trade-off: the *effective subspace rank* selected online and the *projection strength*. Concretely, for each stage we retrieve a query-adaptive basis subset of size m from a pre-built subspace bank, and then apply a confidence-aware soft projection with strength α . A complete experimental report should therefore verify that the main gains are not brittle to these choices, and that the default setting lies on a stable Pareto region rather than a single fragile point.

Sensitivity to top- m . The parameter m controls how much of the jailbreak subspace is removed at inference time. If

Table 4: Sensitivity to query-adaptive rank m . We vary the jailbreak-stage top- m while fixing other settings. We report Average ASR $\downarrow$ over eight jailbreak families on held-out test prompts.

m_{jb}	<i>Mistral-7B</i> Avg ASR $\downarrow$	<i>Llama-3-8B</i> Avg ASR $\downarrow$
4	0.186	0.141
8	0.019	0.015
10	0.017	0.016
12	0.013	0.015
16	0.011	0.008

Table 5: Sensitivity to projection strength α . We vary the jailbreak-stage strength α_{jb} while fixing other settings. We report Average ASR $\downarrow$ over eight jailbreak families on held-out test prompts.

α_{jb}	<i>Mistral-7B</i> Avg ASR $\downarrow$	<i>Llama-3-8B</i> Avg ASR $\downarrow$
0.60	0.038	0.071
0.75	0.020	0.051
0.90	0.012	0.033
0.95	0.011	0.032
1.00	0.011	0.032

m is too small, the intervention under-approximates the jailbreak manifold and leaves residual attack directions, increasing ASR. If m is too large, the projection becomes more aggressive and may start suppressing task-relevant components, which can hurt utility. Table 4 shows a broad plateau around $m \in [8, 12]$ where robustness remains strong. We use $m_{\text{jb}} = 10$ and $m_{\text{safety}} = 8$ as defaults in the main results.

Sensitivity to projection strength α . The strength parameter α controls how strongly we remove the selected subspace component. If α is too small, the projection is insufficient and residual jailbreak directions persist, increasing ASR. If α is too large, the intervention may become overly suppressive and can risk unnecessary refusals or degraded helpfulness. Table 5 indicates that robustness improves quickly as α_{jb} increases from low values, and then saturates around $\alpha_{\text{jb}} \in [0.9, 1.00]$. We adopt $\alpha_{\text{jb}} = 0.90$ and keep $\alpha_{\text{safety}} = 0.55$ fixed in the main setting for a stable robustness-utility tradeoff.

Calibration summary for reproducibility. For transparency, Table 6 summarizes the offline calibration artifacts on *Mistral-7B*, including the energy-based Top-4 layers, the best AUC, and the fused EVT thresholds. We observe that the energy-based separability is saturated across all evaluated jailbreak families (AUC= 1.00), indicating that the chosen layers provide a highly reliable signal for distinguishing jailbreak-style prompts from benign ones under the calibrated setting.

Table 6: Offline calibration summary on *Mistral-7B*. We report Top-4 layers, best AUC, and fused EVT thresholds for safety and each jailbreak family.

Concept	Top-4 layers	AUC(best)	τ_{safety}	τ_{jb}
Safety	[8, 6, 9, 31]	1.00	0.57	–
Base64	[4, 5, 6, 12]	1.00	0.57	0.27
DrAttack	[32, 31, 30, 29]	1.00	0.57	0.28
Flip	[25, 32, 30, 31]	1.00	0.57	0.23
I-GCG	[28, 26, 27, 25]	1.00	0.57	0.32
IJP	[4, 3, 7, 6]	1.00	0.57	0.50
Puzzler	[7, 8, 10, 14]	1.00	0.57	0.17
SAA	[25, 26, 27, 32]	1.00	0.57	0.18
Zulu	[11, 10, 30, 29]	1.00	0.57	0.32

Notably, the selected layers vary substantially by attack family: some are dominated by earlier layers (e.g., Base64 and IJP), whereas others concentrate on late layers (e.g., DrAttack and Zulu), reflecting heterogeneous representation footprints across jailbreak mechanisms. Meanwhile, the fused safety threshold remains stable ($\tau_{\text{safety}} = 0.57$) across attacks, while the jailbreak thresholds span a wider range ($\tau_{\text{jb}} \in [0.17, 0.50]$), suggesting that different attacks induce different degrees of concentration in the learned jailbreak subspaces. These observations motivate a clean separation between *offline calibration* and *online control*: we fix calibration artifacts (layers and thresholds) once per model, and tune only the lightweight inference-time knobs (m, α) to trade off robustness and utility.

Takeaway. Overall, our method is not sensitive to a single brittle hyperparameter choice. A moderate rank m is sufficient to capture the dominant jailbreak directions, and a bounded α schedule prevents over-projection on benign prompts. This supports the completeness of our evaluation and justifies using a single default configuration across models and attack families.

A.3 Model capability analysis.

A practical jailbreak defense must avoid suppressing benign instruction-following behavior. We therefore quantify *over-refusal* by the **false positive rate (FPR)** on held-out **harmless** prompts in our jailbreak-vs-harmless gating task. Since our method is only activated when the gate fires, the benign FPR provides a conservative upper bound on unnecessary refusals. As shown in Table 7, the LEARNED fusion substantially reduces benign false triggers to 1.4% on average, compared to 8.0% for JB-only and 9.9% for OR gating. This indicates that our confidence-aware, learned fusion gate is markedly less prone to spurious activation, thereby preserving normal model utility while maintaining strong jailbreak sensitivity. Notably, the remaining false triggers are concentrated in en-

Table 7: Over-refusal on benign prompts (Mistral-7B). We report the false positive rate (FPR, %) on held-out harmless prompts for different fusion rules in the jailbreak-vs-harmless gate. Lower is better.

Jailbreak family	JB-only	OR	LEARNED (Ours)
Puzzler	10.5	12.7	0.5
SAA	4.1	6.5	0.0
DrAttack	5.0	6.7	0.2
IJP	11.2	12.6	1.7
Base64	12.9	15.2	5.6
Zulu	8.2	10.1	2.0
Flip	4.8	7.1	0.1
I-GCG	7.3	8.5	0.9
Avg.	8.0	9.9	1.4

coding and obfuscation-like patterns such as Base64, which is expected because these inputs share superficial formatting traits with some benign structured text. Nevertheless, Learned still yields a large reduction over heuristic gates. Concretely, JB-only activates the defense whenever the jailbreak gate exceeds its EVT threshold and therefore relies solely on the jailbreak signal. OR activates when the safety gate fires or the jailbreak gate fires, which makes it intentionally conservative. However, this conservativeness increases spurious activations on harmless prompts, which manifests as a higher benign FPR. In turn, the system applies unnecessary interventions and may either refuse benign requests or over-project representations, both of which reduce perceived helpfulness on normal instruction-following queries. In contrast, Learned fusion trains a lightweight decision rule on standardized gate features and uses that rule to better separate jailbreak from harmless prompts. This substantially reduces benign false triggers while retaining high jailbreak sensitivity. For this reason, we adopt Learned fusion as the default gating strategy for subsequent mitigation experiments to balance robustness and utility. Finally, we emphasize that benign FPR measures erroneous gate activations and should be viewed as a conservative proxy for over-refusal. The exact user-facing impact depends on whether the intervention policy uses hard refusal or soft projection, but reducing spurious activations is uniformly beneficial for preserving benign utility.

A.4 Interpretability Case Study

Logging protocol. To interpret which concepts are suppressed, we log the top m retrieved bases at each triggered step and compute the projection residual and subspace energy before and after intervention. For each prompt, we record the trigger layer, token position, the gate score trajectory, and the resulting projection strength α for the safety and jailbreak stages. We also compute token level summaries by aggregat-

ing retrieved basis indices and residual magnitudes across the generation, which reveals which concept directions are repeatedly activated and attenuated.

Token level concept traces. We map the retrieved bases to human readable token anchors by projecting vocabulary embeddings onto basis directions and reporting the highest scoring tokens. These anchors provide a qualitative handle on which semantic directions are being attenuated, and we primarily use them as contrastive signatures between benign prompts and jailbreak prompts. Consistent benign and jailbreak differences support that the banks capture attack correlated directions rather than generic instruction following features. Figure 4 shows aggregated top-token anchors over 50 prompts, where the safety subspace emphasizes refusal and policy-related terms while the jailbreak subspace concentrates on override and prompt-control words.

Case study format. For each case study, we present the original prompt, the triggered layers for safety and jailbreak stages, the gate trajectory, the top m basis anchors, and the model output before and after defense. We include cases from encoding based attacks and optimization based suffix attacks to show that the defense triggers on representation level signatures even when surface cues are obfuscated.

Example case. This prompt uses unfiltered character settings and explicitly requests bypassing policy restrictions, which is a typical instruction injection pattern. With projection mode `last_token`, the jailbreak stage triggers at layer 4 and the safety stage triggers at layer 8, suggesting override intent emerges earlier while harmfulness consolidates at mid layers. After intervention, the jailbreak subspace energy drops from 0.216 to 0.022, corresponding to a 89.8% relative reduction, while the safety subspace energy drops from 0.231 to 0.106, corresponding to a 54.0% relative reduction. This asymmetric reduction indicates substantially stronger suppression on attack correlated directions. The safety and jailbreak subspaces remain disentangled with $\|Q_s^\top Q_j\|_F \approx 0.045$, which supports that the two banks capture distinct concepts rather than redundant instruction following features. Finally, `last_token` and `all_tokens` projection yield nearly identical energy reductions on this prompt, indicating the attack signature is concentrated at the triggering position and can be mitigated with a lightweight last token intervention.

Interpretability of coupled orthogonalization. Figure 5 visualizes the geometric interaction between the safety concept subspace and the jailbreak compliance subspace. Before orthogonalization, the overlap matrices $U^\top V$ exhibit non-trivial structured correlations across all jailbreak families, indicating that the raw jailbreak subspaces share directions

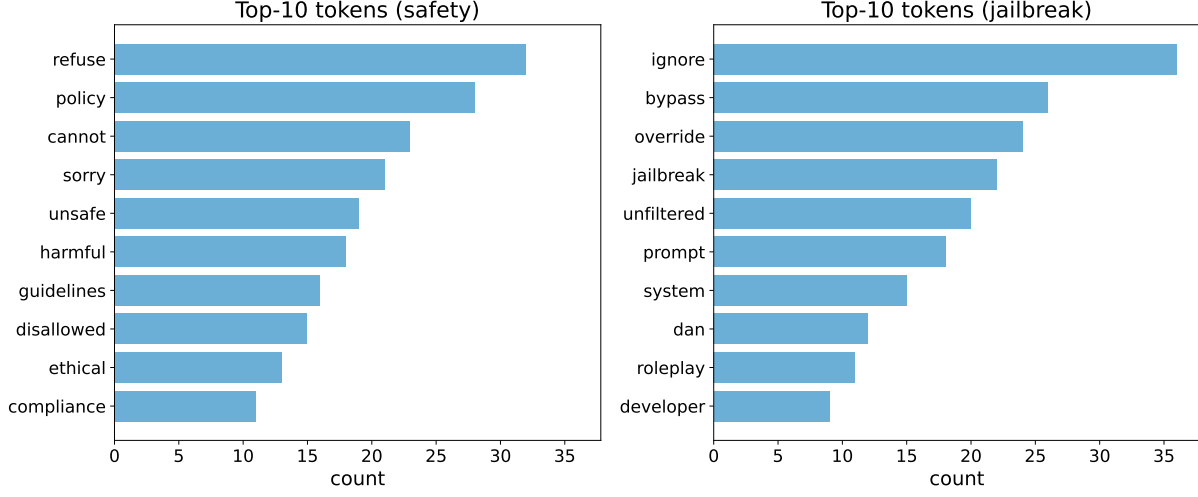


Figure 4: Top-10 token anchors in the retrieved safety and jailbreak subspaces aggregated over 50 prompts (counts).

with safety-related representations. We then apply the coupled orthogonalization step in the offline bank construction, $Q_j \leftarrow \text{QR}((I - Q_s Q_s^\top) Q_j)$, which removes the component of the jailbreak subspace that lies in the span of the safety subspace. After this update, $U^\top V^\perp$ becomes approximately zero across attacks, and the principal angles consistently shift toward 90° . This confirms that the learned jailbreak subspaces can be made nearly orthogonal to safety subspaces, reducing concept interference and supporting more selective inference-time projection.

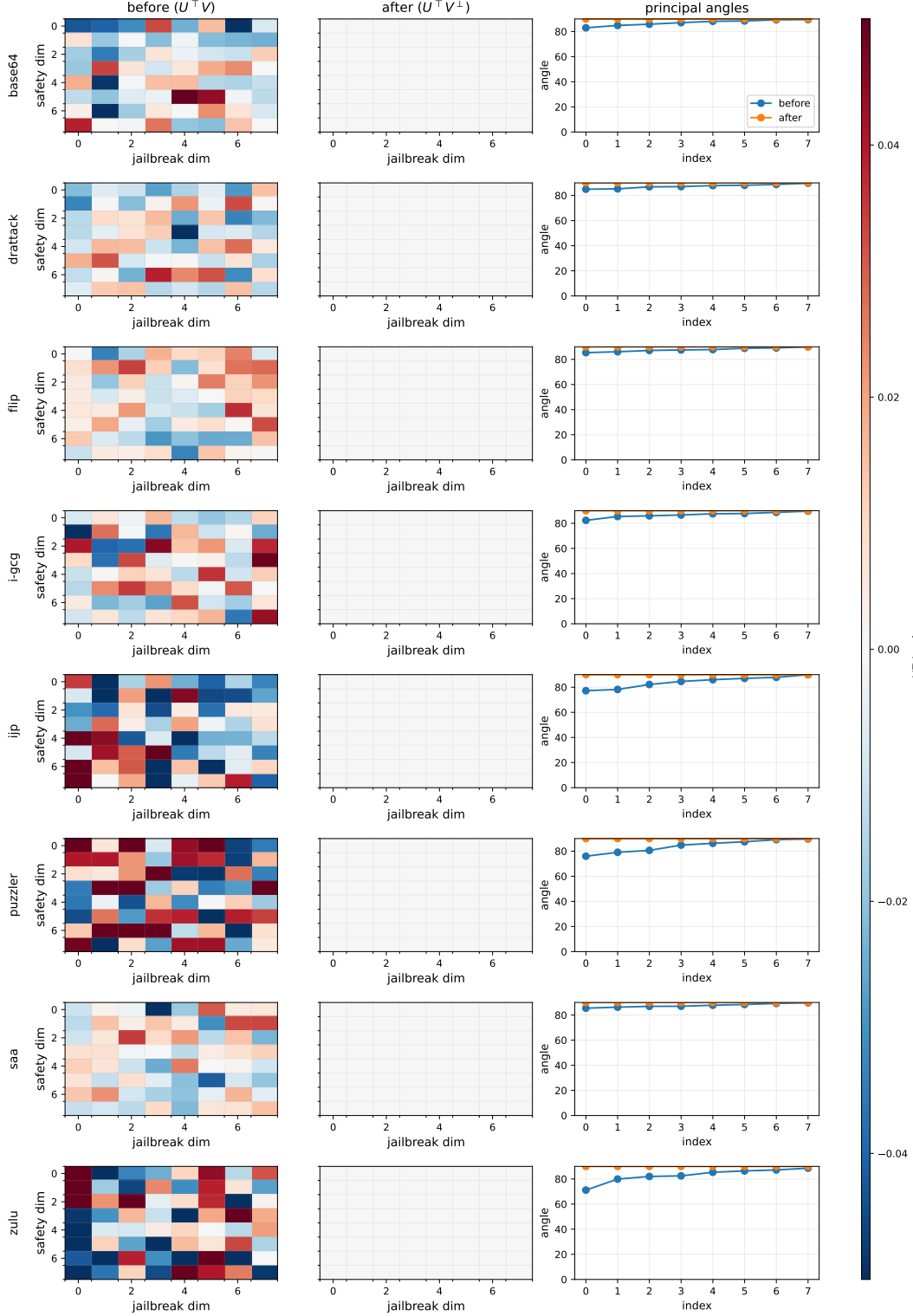


Figure 5: Coupled subspace orthogonalization diagnostics across jailbreak families. Each row corresponds to one attack family at the selected intervention layer. Left: overlap matrix $U^T V$ between the safety subspace basis U and the jailbreak subspace basis V . Middle: overlap matrix $U^T V^\perp$ after orthogonalizing the jailbreak subspace against the safety subspace via $Q_j \leftarrow \text{qr}((I - Q_s Q_s^\top) Q_j)$, which ideally yields near-zero cross-subspace correlations. Right: principal angles between the two subspaces before and after orthogonalization; the x-axis index enumerates principal angles (sorted), and angles closer to 90° indicate stronger orthogonality.